

SSR, SSG & ISR — Next.js

Introduction

SSR (Server-Side Rendering), **SSG (Static Site Generation)**, এবং **ISR (Incremental Static Regeneration)** হলো Next.js-এ webpage render করার বিভিন্ন approach।

মূল পার্থক্য হলো:

HTML কখন তৈরি হবে এবং **data** কতক্ষণ **cache** করে রাখা হবে।

SSR → Request time-এ HTML তৈরি

SSG → Build time-এ HTML তৈরি

ISR → আগে HTML তৈরি + পরে নির্দিষ্ট সময় পর regenerate

1. SSR — Server-Side Rendering

SSR-এ **user**-এর **request** আসার পর **server** HTML তৈরি করে **Browser**-এ পাঠায়।

Flow

User Request



Server



Data Fetch



HTML তৈরি



Browser



Content দেখা যায়



JavaScript loads



Hydration



Interactive

এখানে গুরুত্বপূর্ণ বিষয়:

SSR-এ HTML আগে থেকেই তৈরি করে রাখা থাকে না।

Request আসার পর server সেই request-এর জন্য HTML তৈরি করে।

Example

```
const res = await fetch("http://localhost:3001/products", {  
  cache: "no-store"  
});
```

```
const products = await res.json();
```

cache: "no-store" কী?

cache: "no-store"

মানে:

এই fetch-এর result cache করে রেখো না।

তাই নতুন **request** এলে নতুন করে **data fetch** করা হবে।

User 1



Server → API/DB → HTML

User 2



Server → API/DB → HTML

User 3



Server → API/DB → HTML

কখন SSR ব্যবহার করা হয়?

যখন প্রতিটি **request-এ fresh data** প্রয়োজন।

উদাহরণ:

- User dashboard
- Personalized page
- Frequently changing data

- Request অনুযায়ী আলাদা content
-

2. SSG — Static Site Generation

SSG-তে **Build time**-এ HTML তৈরি করে রাখা হয়।

অর্থাৎ user request করার আগেই page তৈরি হয়ে ready থাকে।

Flow

Build Time



Data Fetch



HTML তৈরি



Static HTML ready



User Request



Ready-made HTML

এখানে:

HTML আগে থেকেই তৈরি করে রাখা থাকে।

Example

```
const res = await fetch("http://localhost:3001/products", {  
  cache: "force-cache"  
});
```

```
const products = await res.json();
```

cache: "force-cache" কী?

cache: "force-cache"

মানে:

Fetch-এর result cache করে রাখা এবং পরবর্তীতে cached result ব্যবহার করা।

Conceptually:

Build/Generation



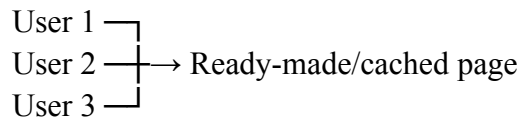
Data fetch



Cache



Static page



কখন SSG ব্যবহার করা হয়?

যখন data খুব ঘনঘন পরিবর্তন হয় না।

উদাহরণ:

- Blog
- Documentation
- Portfolio
- About page
- Static information pages

3. ISR — Incremental Static Regeneration

ISR হলো SSG-এর মতো **static/cached page**, কিন্তু নির্দিষ্ট সময় পর page-টি নতুন data দিয়ে regenerate করা যায়।

সহজভাবে:

ISR = SSG + Periodic Re-generation

Example

```
const res = await fetch("http://localhost:3001/products", {  
  next: {  
    revalidate: 60  
  }  
});
```

```
const products = await res.json();
```

এখানে:

revalidate: 60

মানে cached data প্রায় **60 seconds** পর stale হলে নতুন data দিয়ে regenerate করা যেতে পারে।

Flow

Build

↓

Static HTML

↓

User gets static page

↓

60 sec পরে নতুন request

↓

নতুন data fetch

↓

নতুন HTML generate

তাই প্রতিটি request-এ নতুন করে database/API থেকে data আনতে হয় না।

কখন ISR ব্যবহার করা হয়?

যখন data পরিবর্তন হয়, কিন্তু প্রতিটি **request-এ fresh data** প্রয়োজন হয় না।

উদাহরণ:

- News website
- Product catalog
- Blog
- E-commerce products
- Frequently updated content

SSR vs SSG vs ISR

বিষয়	SSR	SSG	ISR
HTML কখন তৈরি হয়?	Request time	Build time	Build time + regeneration
HTML আগে থেকেই ready থাকে?	✗	✓	✓
Data কখন fetch হয়?	Request অনুযায়ী	Build/generation time	Revalidation অনুযায়ী
Cache	সাধারণত no-store	force-cache	revalidate
Freshness	সবচেয়ে বেশি	Build-এর data	Periodic update
প্রতিটি request-এ নতুন rendering?	✓	✗	✗ সাধারণত
Use case	Dynamic data	Static data	Periodically changing data

Type	HTML কখন তৈরি হয়?	Data কখন fetch হয়?	সহজভাবে
SSR	প্রতিটি request-এর সময়	প্রতিটি request-এর সময়	নতুন request → নতুন page
SSG	Build time-এ	Build time-এ	আগে থেকেই page তৈরি
ISR	Build time-এ + পরে নির্দিষ্ট interval-এ update	নির্দিষ্ট interval অনুযায়ী	SSG + automatic update

Fetch Options

1. `cache: "no-store"`

```
fetch(url, {  
  cache: "no-store"  
});
```

মানে:

Cache করো না



প্রতিবার request এ নতুন data

এটি dynamic/request-time rendering-এর জন্য ব্যবহার করা হয়।

2. `cache: "force-cache"`

```
fetch(url, {  
  cache: "force-cache"  
});
```

মানে:

Data cache করো



Cached data reuse করো

Static/cached data-এর ক্ষেত্রে ব্যবহার করা যায়।

3. `next: { revalidate: 60 }`

```
fetch(url, {  
  next: {  
    revalidate: 60  
  }  
});
```

মানে:

Data cache করো



60 sec পরে stale



প্রয়োজনে নতুন data দিয়ে regenerate

এটাই ISR-এর মূল concept।

গুরুত্বপূর্ণ সতর্কতা

cache: "no-store" দেখলেই সবসময় পুরো route-এর behavior-কে শুধু "SSR" বলে বোঝানো ঠিক নয়। একইভাবে **force-cache** দেখলেই প্রতিটি পরিস্থিতিতে "SSG" ধরে নেওয়াও ঠিক নয়।

Next.js-এর rendering behavior-এর ওপর আরও কিছু বিষয় প্রভাব ফেলতে পারে, যেমন:

- **cookies()**
- **headers()**
- Dynamic APIs
- Route configuration
- অন্যান্য dynamic behavior

তাই মূল concept হিসেবে মনে রাখো:

SSR → Request-time rendering

SSG → Build/static generation

ISR → Static generation + Revalidation
